

Our workflow service, as Temporal would want to see it.

THE ASK, IN ONE SENTENCE

Assess the effort, risk and sequencing of porting [ms-bpm](#) from an embedded Camunda 7 engine to [workflow code on the Temporal Java SDK](#), on the Temporal Cloud account BFI already runs.

WHY THIS DECK EXISTS

We measured our own system before asking. Every element in our models is counted and mapped, every engine call in our code is listed, the production numbers are real, and a year of running Temporal for our other platform is written down. The six markdown files that come with this deck hold the detail.

One Spring Boot service, one embedded engine, one database, every retail loan we write

A consumer-finance lender in Indonesia. `ms-bpm` takes a loan application from lead to go-live: survey, credit bureau, scoring, underwriting, approval, signing. It is the system of record for applications in flight.

135_k LOAN APPLICATIONS STARTED A MONTH	541_k APPLICATIONS OPEN RIGHT NOW, TAIL UP TO ONE YEAR	53 BPMN MODELS AND 3 DMN TABLES, ALL TO BECOME JAVA	272 JAVA DELEGATE CLASSES BEHIND 483 SERVICE TASKS, FUTURE ACTIVITIES
---	--	---	---

TWO GENERATIONS OF MODEL, BOTH IN SCOPE

Legacy, per product: five large flat processes (NDF4W, NDF2W, repeat-order, Sharia, unsecured) carry about 94% of volume and 78% of the BPMN by line. **Unified:** one spine that calls 32 child processes, one per domain step, where new products land. Product routing is a map in configuration, not a diagram decision.

WHAT SITS AROUND THE ENGINE, AND STAYS

About thirty microservices called from delegates. 238 JPA entities and 271 tables in PostgreSQL. Three in-house operator consoles (764,000 lines of React) that **never see an engine task id**. Keycloak for identity. All of it stays where it is; only the orchestration tier moves.

The stack, and the date that brought us to you

COMPONENT	VERSION	NOTE
Camunda Platform 7	7.23.0 Community Edition	engine, webapp and REST starters embedded in the JVM; DMN engine; JUEL expressions
Spring Boot / Framework	3.5.16 / 6.2.19, moving to 4	Spring Cloud 2025.0, 113 Feign clients to other services
Java	17	distroless image, container-aware JVM at 80% of the memory limit
Database	PostgreSQL 16.14 on Cloud SQL	engine tables and 271 application tables share one schema and one transaction
Platform	GKE, Helm, ArgoCD	2 replicas conventional (4-8 CPU, 8-16 GiB each), 4 replicas Sharia; Istio and Cloud SQL proxy sidecars
Identity	Keycloak OIDC	custom read-only identity provider plugin for Cockpit; engine user tables empty
Observability	Datadog APM	HTTP, JPA and outbound calls traced; no engine-level metrics exported

Camunda 7 Community Edition ended in October 2025. We run the last public build. We are assessing more than one route off it; this package asks you to assess the Temporal one.

TWO PRODUCTION DEPLOYMENTS, SAME CODE

Conventional and **Sharia**, each with its own database and promotion pipeline. Sharia is small (86 applications in 90 days) but regulated separately. Namespace design has to cover both.

What is in the 53 models, and our first mapping of each to Temporal

BPMN TODAY	COUNT	TEMPORAL CONSTRUCT, OUR READING	DIFFICULTY
Service tasks, all <code>delegateExpression</code>	483	activity methods; delegate bodies port near-verbatim	low, largest block
Exclusive gateways / JUEL conditions	513 / 659	<code>if</code> and <code>switch</code> in workflow code	low per item, high in volume
Call activities, propagate-all	56	child workflows or in-process calls	low
User tasks	96	<code>Workflow.await</code> on a signal or update	medium: the one-year tail
Retry cycles, 31 patterns	186	<code>RetryOptions</code> per activity	low: policy already exists
<code>BpmnError</code> sites / error codes	88 / 114	non-retryable <code>ApplicationFailure</code> subtypes	medium
Escalation end + boundary events	108 + 82	no equivalent: typed returns or exceptions, child to parent	high
Link throw + catch events	146 + 48	no equivalent: a <code>goto</code> into shared handlers, becomes structured flow	high
Embedded sub-processes	154	inlined blocks or child workflows	medium
Timers / DMN tables	12 / 3	<code>Workflow.sleep</code> / plain Java	low

No script tasks, no message or signal events, no multi-instance, no listeners, no external tasks. The surface is simpler than the file count suggests, and the hard part is two idioms, not fifty.

The hard part is concentrated in two legacy files, which changes the sequencing question

ELEMENT	UNIFIED	LEGACY	LEGACY SHARE
BPMN lines	8,540	30,294	78%
Service tasks	82	401	83%
Exclusive gateways	96	417	81%
User tasks	24	72	75%
Embedded sub-processes	7	147	95%
Error definitions	5	205	98%
Link events	35	159	82%
Escalation events	137	53	28%

WHAT THIS MEANS

The unified spine is 37 files and 8,540 lines, with per-product variation already held as **data** in configuration tables rather than in the diagram. Porting it is a bounded job. The two legacy monoliths, `ndf4w.bpmn` and `ndf2w.bpmn`, are 46% of all BPMN on their own and hold 212 of the 384 escalation and link elements. They also use errors, not escalations, as the child-to-parent channel, so a direct port designs **two** return idioms.

THREE ORDERINGS WE WANT YOUR VIEW ON

Direct: port everything. **Consolidate first:** move legacy products onto the unified spine inside Camunda, then port one spine; removes 78% of the BPMN from the port but migrates the biggest product twice. **Spine first:** port the spine as the pilot on about 5% of volume, then move each legacy product straight onto the Temporal spine, writing the long-tail activities once.

Thin at the API, deep in one place: the shared transaction

0.4%

OF 499,000 JAVA LINES TOUCH AN ENGINE API:
ABOUT 2,000 LINES TO REWRITE

1,917

SETVARIABLE SITES; 995 GETVARIABLE .
BECOMES TYPED WORKFLOW STATE

0

HISTORY QUERIES, EXTERNAL TASKS, MESSAGE
CORRELATION, BUSINESS KEYS

48

USER-TASK COMPLETE SITES AND 24 CLAIM
SITES, THE WHOLE HUMAN-TASK SEAM

ENGINE SERVICE	CALLS	BECOMES
TaskService	96	signals or updates on the workflow client
RuntimeService	61	start 14, query 14, variables 21, modification 5 , delete 4, incidents 3; client start, visibility queries, and whatever replaces modification
ManagementService	1	retry from our own API; activity retry handling
Engine internals	9 files	a last-retry aspect (activity attempt info), a Cockpit identity plugin (dropped), one utility

THE ONE DEEP DEPENDENCY

A delegate updates our JPA entities and the engine updates ACT_RU_* in the same PostgreSQL transaction. If the delegate throws, both roll back together. 260 asyncAfter boundaries were placed with that in mind. As activities, the database write and the activity completion are two operations, and every one of 483 activities needs to be safe to run again. We think most of it can live in one base activity class and an idempotency key. We want your view.

Long-running instances at volume: the numbers that size the move

400_k

TOP-LEVEL INSTANCES IN 90 DAYS; 1.28M INCLUDING CALL-ACTIVITY CHILDREN

7,943

APPLICATIONS ON THE BUSIEST DAY (23,865 ENGINE INSTANCES); 1,347 IN THE BUSIEST HOUR

1.10_M

ENGINE INSTANCES OPEN; 1.04M OPEN USER TASKS; 46M RUNTIME VARIABLES

5–15_M

ACTIONS A MONTH, OUR ROUGH ESTIMATE FROM 1.4M ACTIVITY ROWS A DAY; PLEASE REPLACE IT

PROCESS KEY, 90 DAYS	STARTED		OPEN	MEAN
NDF2W	253,728		100,633	54 h
NDF4W	61,930		17,298	99 h
OPERATION	21,999		2,395	25 h
Unified_Process_Main_Workflow	20,092		9,079	142 h
NDF4W_RO	11,713		2,592	43 h

THERE IS NO DRAIN WINDOW, AND NO INSTANCE MIGRATION TOOL

540,824 top-level applications are open. 408,638 started more than 90 days ago, 59,710 more than a year ago. Nothing moves a Camunda instance into a Temporal execution. So the shape is either **route new applications to Temporal and let Camunda run off for a year**, or **reconstruct state and start matching executions**. We want your view on both, and on what a workflow parked for a year costs in history.

We are not a new customer: a year of production in Go, and a Java pilot in review

8 GO SERVICES ON THE TEMPORAL GO SDK, VERSIONS 1.34 TO 1.48, ON OUR OTHER ORIGINATION PLATFORM	49_k RUNNING EXECUTIONS IN THE PRODUCTION NAMESPACE; ABOUT 5,000 STARTS A DAY	22_k PEAK RUNNING PER VERSION AT A HANDOVER; ONE TASK QUEUE PER VERSION	1.39 JAVA SPRING BOOT STARTER IN A SIBLING BRAVO SERVICE: UPDATE-WITH-START, STEPS FACADE, TYPED FAILURES
--	---	---	---

FIVE LESSONS WE BRING TO THE CALL

Worker memory is the sticky cache: 0.75 MB per cached workflow, one worker type OOM-killed until we sized it. **Retry ceilings:** a 60-second maximum interval turned one outage into thousands of retries an hour; now 15 minutes. **Search attributes** were never defined, so “which loans are stuck at survey” is answered from our database, not Temporal.

Version retirement needed a runbook for orphaned children. **Trace spans** miss 30–70% of long-running completions; use Temporal’s own counts.

WHAT THE JAVA PILOT ALREADY SETTLED

One workflow type per loan product from a template with a router. Business rejections as non-retryable `ApplicationFailure` mapped to the exact HTTP error body the consoles expect. A 30-day `Workflow.await` for a document wait. Worker in-process with the API pods, dark unless a target is configured. gRPC and protobuf pinned ahead of the Google BOM. **What it has not done:** a workflow that lives for days with human waits, at 135,000 starts a month. That is the gap this assessment closes.

Twelve things we ask Temporal to assess, in writing

1	The element mapping on slide 4, corrected. Especially the 190 escalation and 194 link events, child workflow versus in-process call, and the 78 configuration-reading conditions against determinism.	WE SUPPLY MODELS UNDER NDA
2	The human-task pattern. Signal, update or async-completed activity for 96 wait points with a one-year tail; keeping our task tables and the workflow consistent; history size and when Continue-As-New is needed.	DESIGN
3	In-flight strategy. Route-new-and-run-off versus reconstruct-and-start, for 541k open applications. What other customers did at this scale.	DECIDES THE SHAPE
4	Sequencing. Direct, consolidate first, or spine first (slide 5). Your recommendation with reasons.	DECIDES THE TIMELINE
5	Idempotency and the database. The pattern for 483 activities that relied on a shared transaction; how much is generic; outbox or idempotency keys at this volume.	LARGEST RISK ITEM
6	Operations and versioning. What replaces Cockpit, which search attributes, the equivalent of process instance modification, and the versioning model for year-long workflows given our per-version task-queue experience.	OPS STAFF
7	Sizing, history and audit. A proper Actions estimate; Java worker fleet sizing with the sticky-cache lesson; one or two namespaces; history retention and export for a 90-day audit floor; data residency from Singapore for an Indonesian lender.	PLATFORM
8	SDK, testing, effort and delivery. Spring Boot 3.5 and 4 support; the parity harness with replay and time-skipping tests and a dual run; a phased effort estimate; what Temporal takes on versus our squads; training for Java engineers next to a Go team that already knows Temporal.	PEOPLE AND PLAN

What does not move, and what the port must not break

REGULATED WORKLOAD Auditability and retention are floors Loan origination records for a supervised lender. 90 days of process history is the minimum, queryable after the workflow closes, and decisions must stay explainable after the fact.		NO DRAIN WINDOW Camunda runs alongside for a year Half a million open applications with a one-year tail. Two engines in production under any plan; the application tables are shared, so outcomes can be diffed in one database.	ENDPOINT CONTRACTS 764,000 lines of console must not change The consoles call domain verbs and never see a task id. Preserving those contracts is a hard constraint of the port, and the consoles' 829 tests become the boundary regression suite.
PLATFORM Google Cloud, Temporal Cloud Singapore GKE, Cloud SQL PostgreSQL 16, Keycloak. Data residency for an Indonesian lender needs a current answer.	TWO DEPLOYMENTS Conventional and Sharia Same code, separate databases and pipelines, separate regulatory treatment. One or two namespaces.	TEAM Java squads, a Go team next door Deep Camunda 7 habits, one Java pilot, a year of Go production experience to draw on, thin automated coverage at the process layer.	WHAT WE CAN SHARE, AND WHAT WE CANNOT Can: all 53 BPMN and 3 DMN files under NDA, the Java pilot's design notes, anonymised database statistics, read access to our existing namespaces, and a read-only walk-through of the code with an engineer. Cannot: production data, credentials, or customer records. Not in this package by choice: internal cost figures, our contract terms, and comparisons with other vendors. Those are a separate conversation.

Four weeks from package to written assessment

WEEK 1 Walk-through One call. We take you through this package and the six files, agree the model hand-over and the NDA, and name one technical contact on each side. The Go team that runs LORA joins.	WEEKS 1-2 Analysis You read the models, the integration file and the footprint, and look at our existing namespaces. We answer questions within a day and give a code walk-through if it helps.	WEEKS 3-4 Written assessment The corrected mapping, the recommended sequencing and in-flight strategy, effort ranges, sizing, and the risks you see. The questions on slide 9 answered in writing.	WEEK 4 Review A second call with our engineers, the Go team, and the people who operate the system. Then we decide, and the commercial conversation starts from a shared technical picture.
---	---	--	---

A	Reference customers who ported a BPMN-based origination system with long human waits to Temporal at 100,000-plus starts a month.	HELPFUL
B	Sequencing and in-flight answer first. Slide 9, items 3 and 4, shape how we read everything else and can come before the full analysis.	WEEK 1
C	Spring Boot 4 and Java SDK facts for the 3.5 and 4 lines, so the pilot pattern can be confirmed for <code>ms-bpm</code> early.	WEEK 1

HOW WE WILL BOTH KNOW IT WORKED

We can put the routes side by side with effort, risk and timeline from the same evidence. The migration plan starts from element counts, production numbers and a year of our own Temporal operations that both sides have already agreed, rather than from a discovery phase. And the first workshop is about our system, not about a slide template.